System Analysis & Design

E-Commerce Data Integration & Analytics Pipeline

Digital Egypt Pioneers Initiative (DEPI) - Data Engineering Track

Delivery date: 6 November 2026

1. Problem Statement & Objectives

1.1 Problem Statement

An e-commerce company receives product, customer, order and transaction data from different sources: CSV files, an external API and a product website. The data has duplicates, missing values, inconsistent formats and changing records. Analysts need a reliable central dataset for sales reporting.

1.2 Objectives

- Build an automated end-to-end data pipeline that collects data from many sources.
- Clean and transform the data, then store it in a relational database.
- Prepare an analytical data model (star schema) and SQL views for sales reporting.
- Run the whole process with Apache Airflow, with logging, retry and error handling.

2. Use Case Diagram & Descriptions

The system has two actors: the Data Engineer, who runs and monitors the pipeline, and the Data Analyst, who uses the final data.

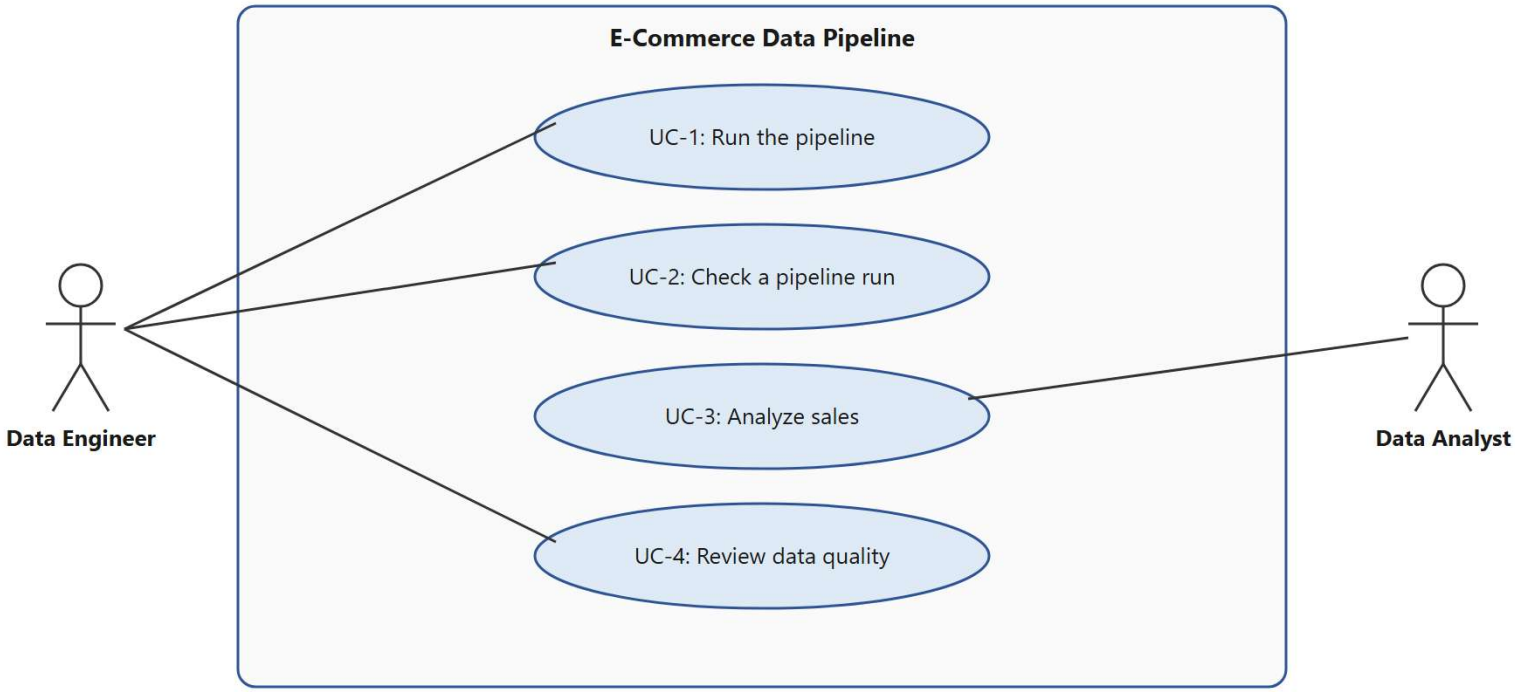


Figure 1: Use case diagram

2.1 Use Case Descriptions

Use case	Actor	Precondition	Main steps	Result
UC-1: Run the pipeline	Data Engineer	Airflow is running and the input files exist	1. Open Airflow. 2. Trigger the DAG. 3. Airflow runs Extract, Validate, Transform, Load and Quality Check.	Clean data is loaded into the database
UC-2: Check a pipeline run	Data Engineer	A DAG run exists	1. Open the DAG run. 2. See the status of each task. 3. Open the log of a failed task.	The engineer knows if the run succeeded or why it failed
UC-3: Analyze sales	Data Analyst	Data is loaded into the database	1. Open the database. 2. Run a SQL view or query on the star schema.	Sales results for reports
UC-4: Review data quality	Data Engineer	The Quality Check task has finished	1. Open the Quality Check result. 2. Read the checks that passed or failed.	Confirmation that the final data is correct

3. Functional & Non-Functional Requirements

The full list of requirements is in the document 2026-10-16 - 3 - Requirements Gathering. This is a short summary:

System capabilities (Functional)

- Get product data from a REST API and customer/order data from CSV/JSON files (FR-1, FR-2).
- Validate the data, handle missing values and duplicates, and standardize data types and formats (FR-3 to FR-6).
- Join and transform the data using Pandas (FR-7).
- Store the data in a star schema and load it incrementally (FR-8, FR-9).
- Provide SQL views and indexes, and run a data-quality check (FR-10 to FR-12).
- Run everything through an Airflow DAG with logging and retry (FR-13, FR-14).

Constraints (Non-Functional)

- Reliability, data quality, performance, usability, maintainability, observability, portability and security (NFR-1 to NFR-8).

4. Software Architecture

Architecture style: a batch ETL pipeline. The data moves in one direction through separate stages (Extract, Validate, Transform, Load, Quality Check). Each stage is a separate Python module, and Apache Airflow runs the stages in order.

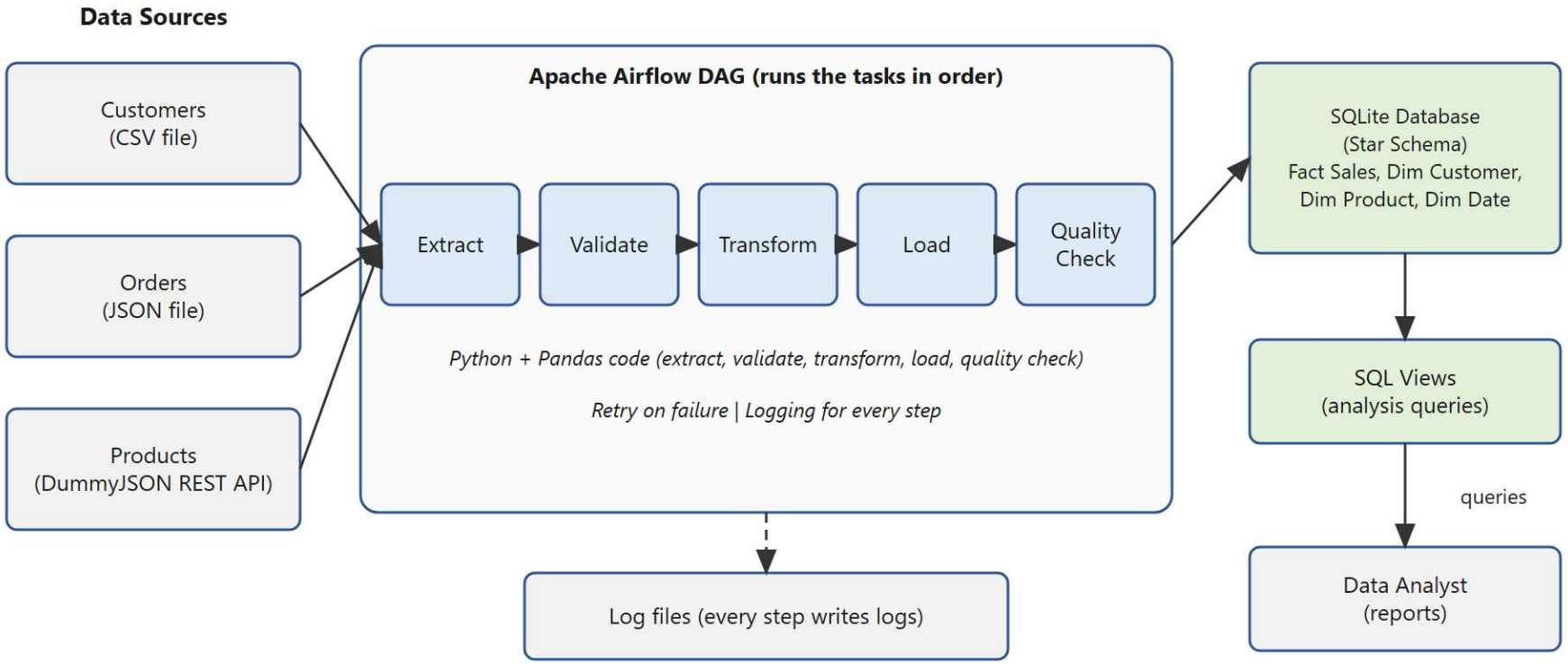


Figure 2: High-level architecture

Component	Responsibility
Data sources	Customers CSV file, Orders JSON file and the DummyJSON REST API (products).
Airflow DAG	Runs the five tasks in order, retries failed tasks and shows the run status.
Python / Pandas modules	Contain the code of each stage.
SQLite database	Stores the star schema (Fact Sales, Dim Customer, Dim Product, Dim Date).
SQL views	Ready queries for common sales analysis.
Log files	Every step writes logs to find problems quickly.

5. Database Design & Data Modeling

5.1 ER Diagram (Star Schema)

The database is a star schema: one fact table (fact_sales) in the center, connected to three dimension tables.

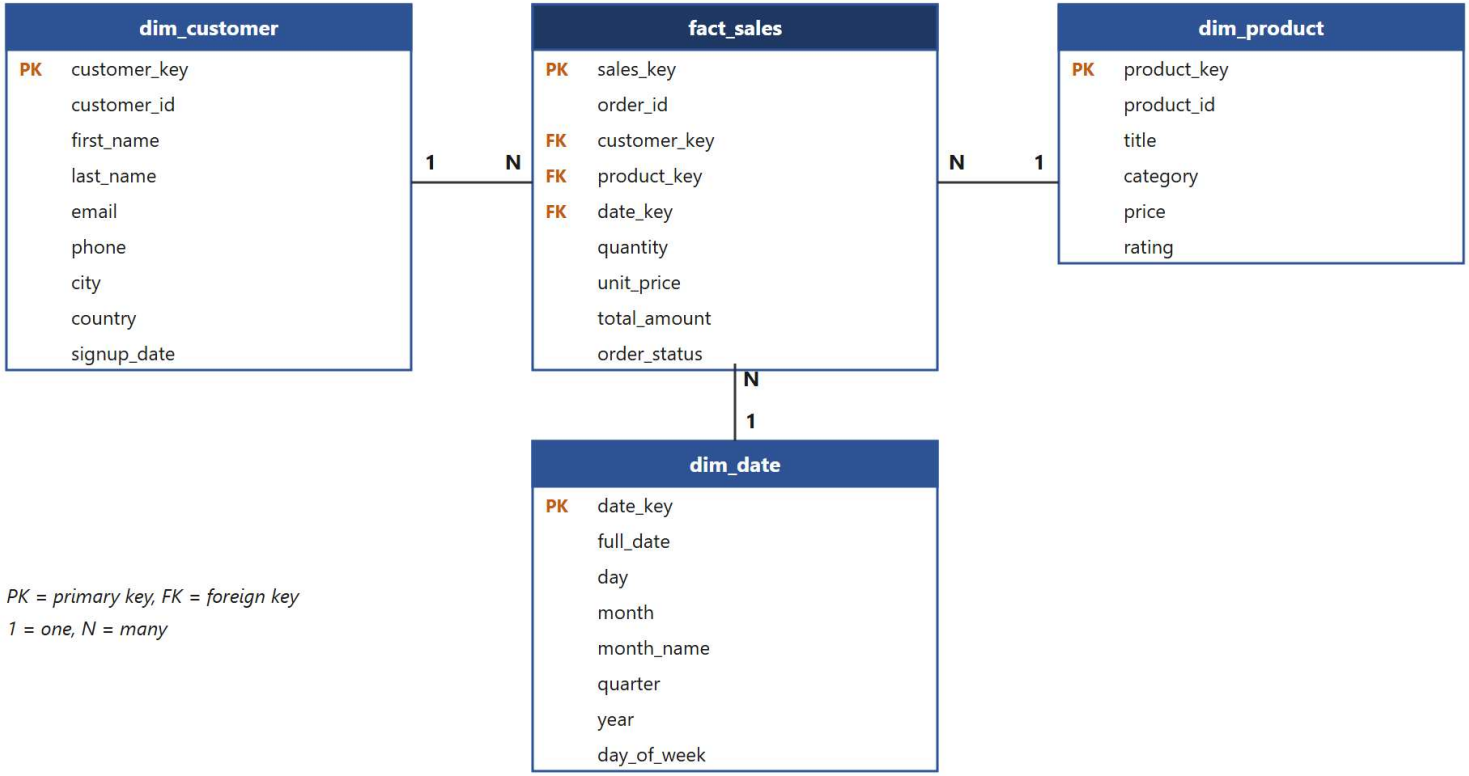


Figure 3: Entity-Relationship diagram

5.2 Logical Schema

Table	Primary key	Foreign keys	Description
dim_customer	customer_key	-	One row per customer
dim_product	product_key	-	One row per product
dim_date	date_key	-	One row per calendar date
fact_sales	sales_key	customer_key (dim_customer), product_key (dim_product), date_key (dim_date)	One row per product in an order; stores quantity, price and total amount

Normalization: the dimension tables are kept denormalized on purpose (a star schema), so the analysis queries need only simple joins. The fact table stores keys and measures only, with no repeated customer or product text.

5.3 Physical Schema (SQLite)

dim_customer

Column	Type	Key / Constraint	Note
customer_key	INTEGER	PRIMARY KEY	Surrogate key (auto number)
customer_id	INTEGER	UNIQUE, NOT NULL	Customer id from the source file
first_name	TEXT		
last_name	TEXT		
email	TEXT		Lower case, trimmed
phone	TEXT		One standard format
city	TEXT		
country	TEXT		
signup_date	TEXT		Format YYYY-MM-DD

dim_product

Column	Type	Key / Constraint	Note
product_key	INTEGER	PRIMARY KEY	Surrogate key (auto number)
product_id	INTEGER	UNIQUE, NOT NULL	Product id from the API
title	TEXT		
category	TEXT		
price	REAL		Current price
rating	REAL		Average rating

dim_date

Column	Type	Key / Constraint	Note
date_key	INTEGER	PRIMARY KEY	Format YYYYMMDD, for example 20260115
full_date	TEXT	NOT NULL	Format YYYY-MM-DD
day	INTEGER		
month	INTEGER		
month_name	TEXT		
quarter	INTEGER		
year	INTEGER		
day_of_week	TEXT		

fact_sales

Column	Type	Key / Constraint	Note
sales_key	INTEGER	PRIMARY KEY	Auto number
order_id	INTEGER	NOT NULL	Order id from the source file
customer_key	INTEGER	FOREIGN KEY	References dim_customer
product_key	INTEGER	FOREIGN KEY	References dim_product
date_key	INTEGER	FOREIGN KEY	References dim_date
quantity	INTEGER		
unit_price	REAL		Product price at the time of loading
total_amount	REAL		quantity x unit_price
order_status	TEXT		

5.4 Indexes, Views and Incremental Loading

Indexes (on the fields that are queried often):

- fact_sales: customer_key, product_key, date_key and order_id
- dim_customer: customer_id
- dim_product: product_id

SQL views (for common analysis questions):

View	Answers the question
v_sales_by_category	How much did each product category sell?
v_sales_by_month	What are the total sales in each month?
v_top_customers	Who are the customers who spent the most?

Incremental loading rule: when the Load task runs, it adds to fact_sales only the orders whose order_id is not already in the table. The dimension tables add only the new customers and products.

6. Data Flow & System Behavior

6.1 Data Flow Diagram (DFD)

Level 0 (context diagram): the whole system as one process with the outside parties that send data to it or receive data from it.

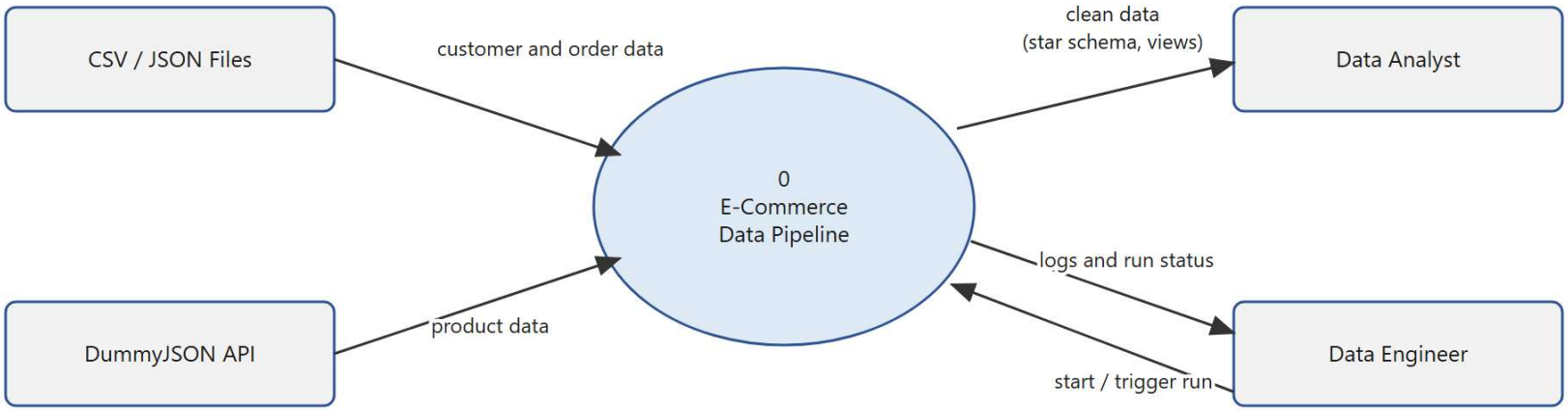


Figure 4: DFD level 0 (context diagram)

Level 1: the system is split into its five processes, with the data stores D1 (database) and D2 (log files).

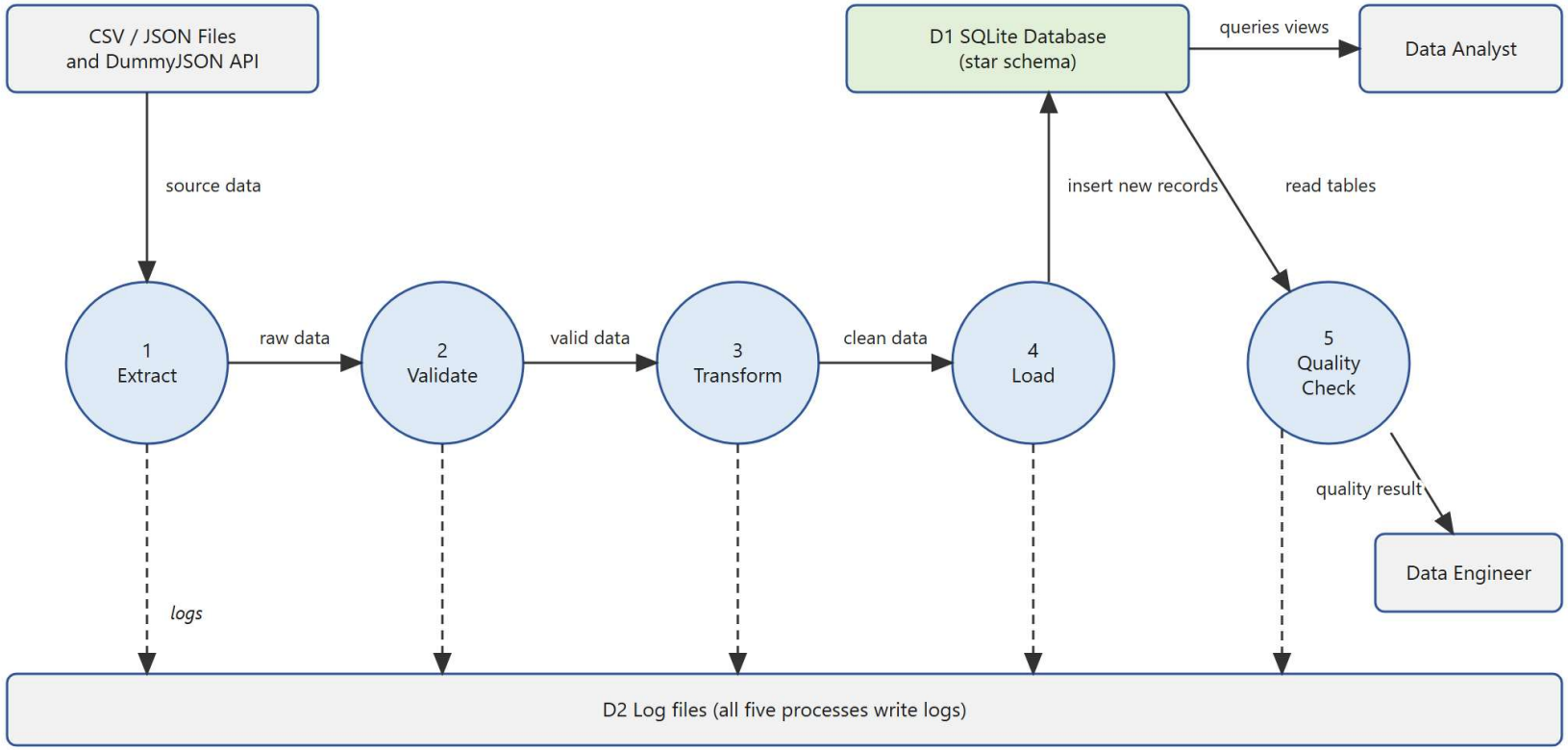


Figure 5: DFD level 1

6.2 Sequence Diagram

The order of messages when the Data Engineer triggers a pipeline run.

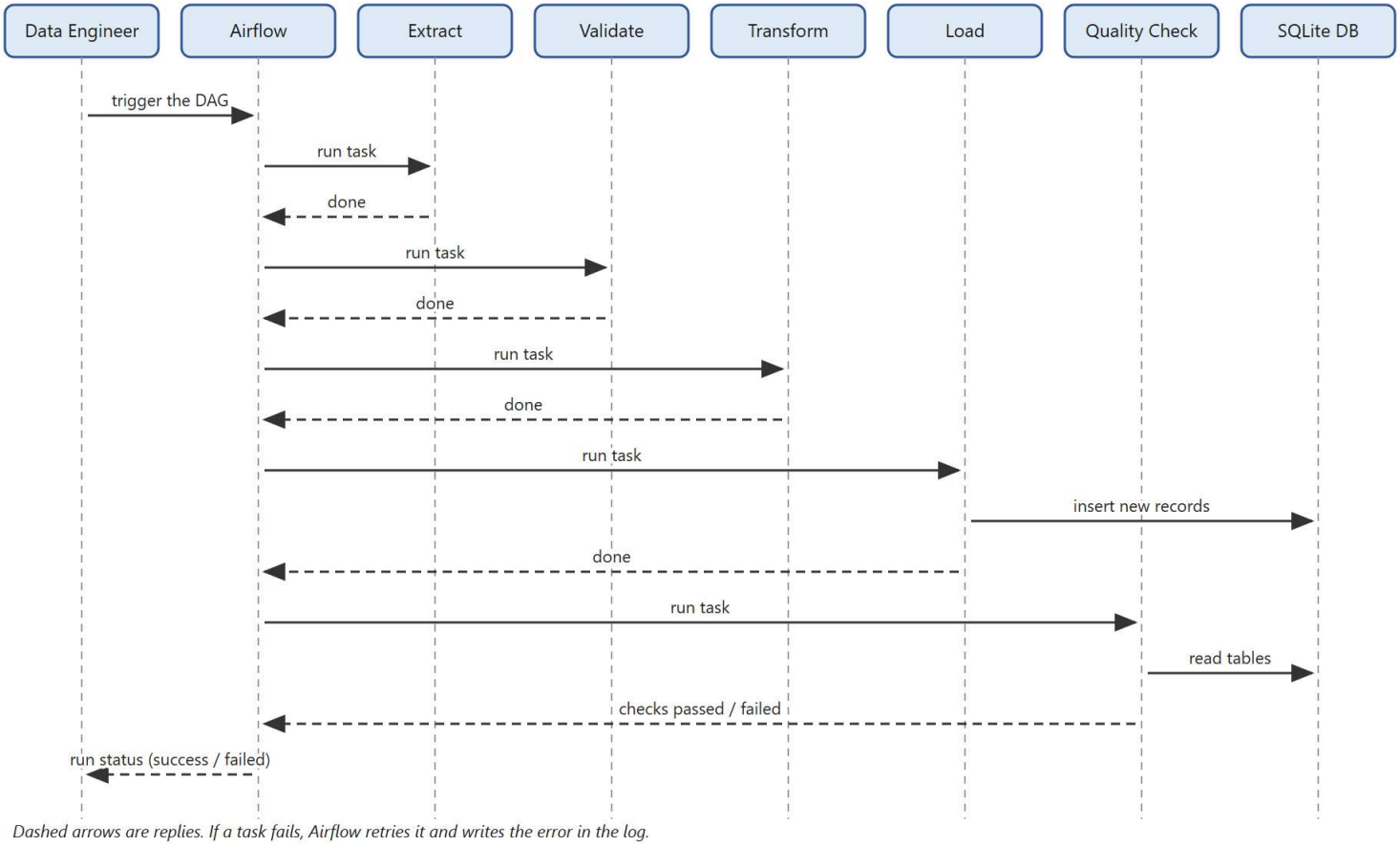


Figure 6: Sequence diagram of one pipeline run

6.3 Activity Diagram

The workflow of one pipeline run, including what happens when the data is not valid or the quality checks fail.

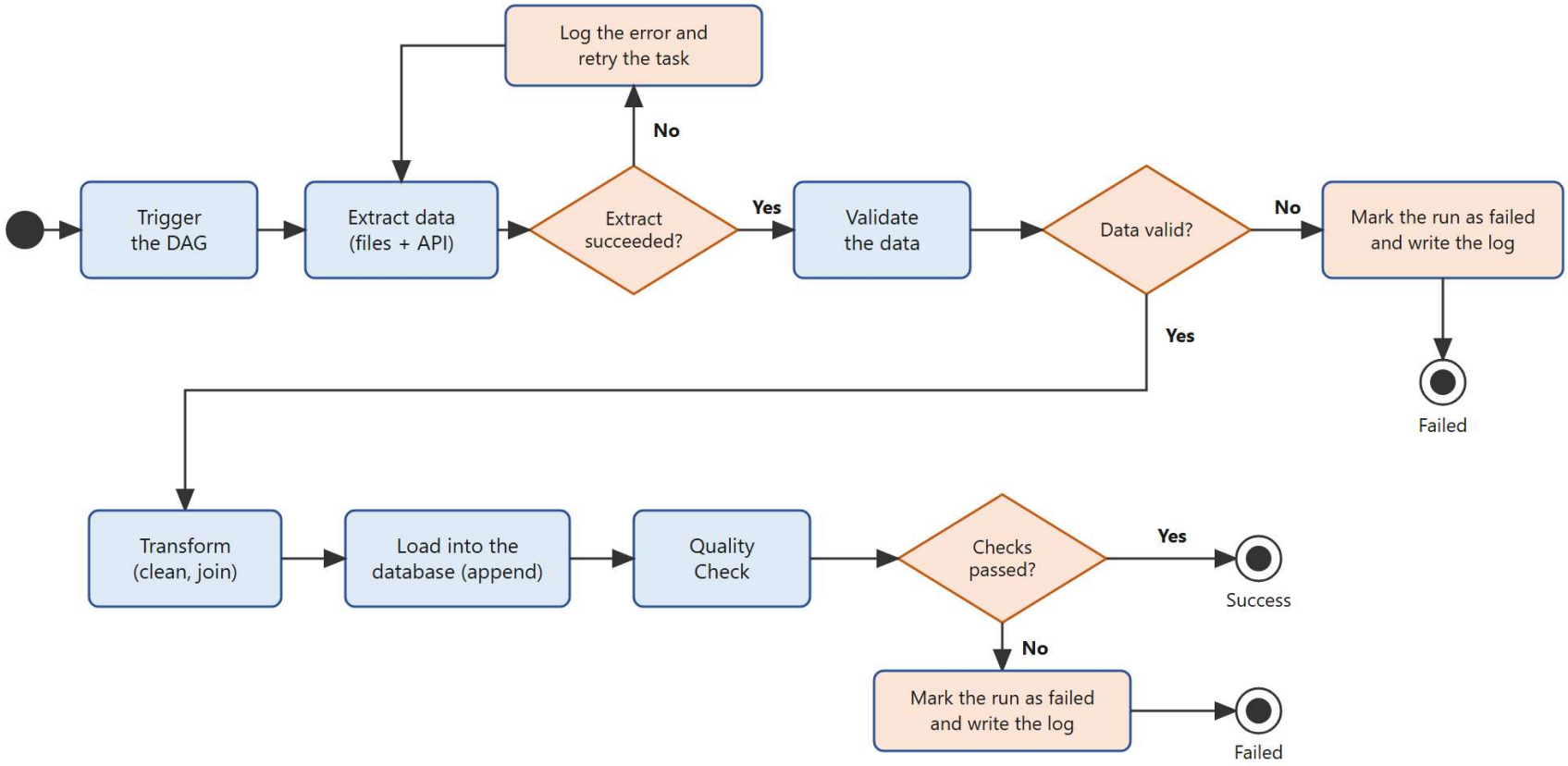


Figure 7: Activity diagram

6.4 State Diagram

The states that one Airflow task goes through during a run.

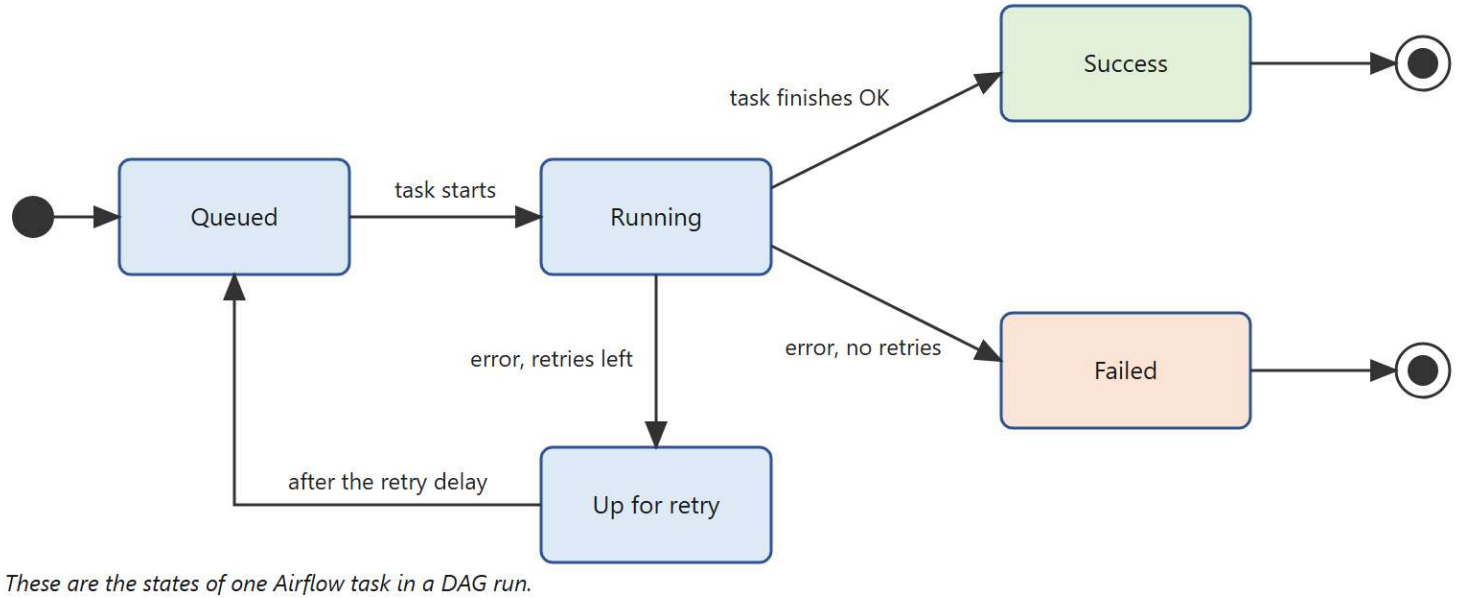
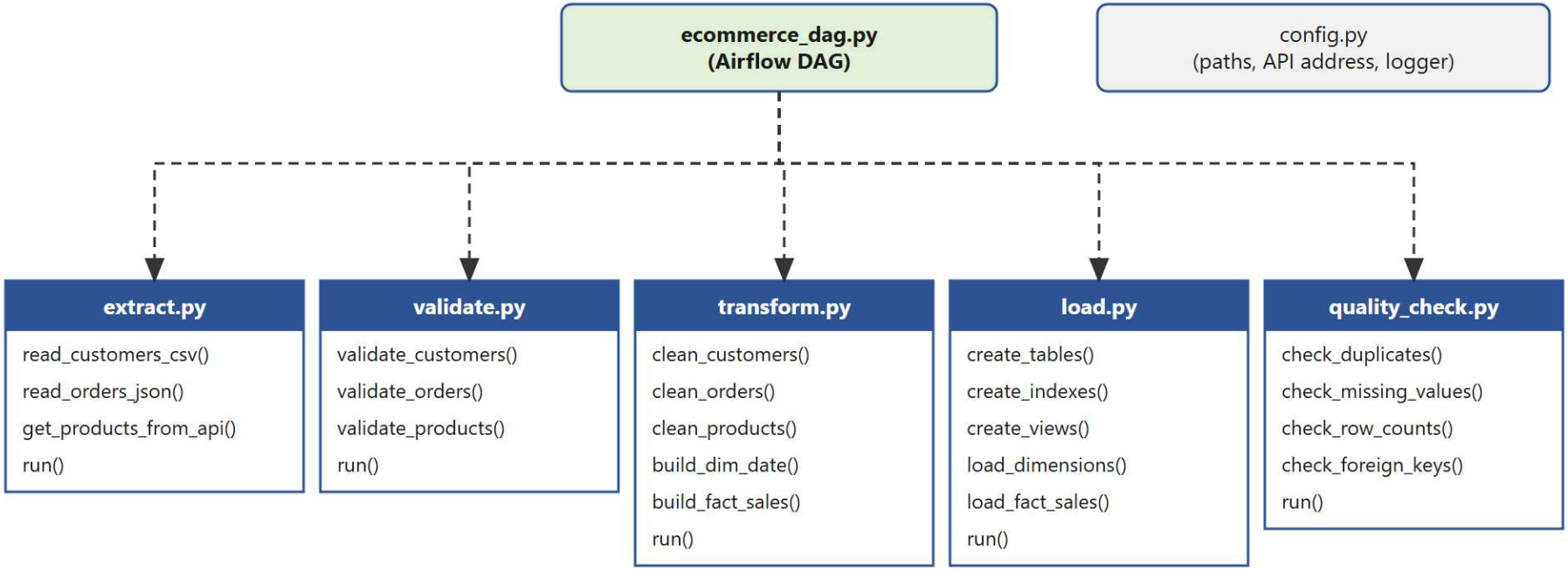


Figure 8: State diagram of an Airflow task

6.5 Class Diagram

Our code is written as simple Python functions grouped in modules. Each module is shown as a box with its functions.



The DAG calls the run() function of each module (dashed arrows); run() uses the other functions of the module. All modules use config.py.

Figure 9: Class (module) diagram

7. System Deployment & Integration

7.1 Technology Stack

Layer	Technology	Use
Language	Python 3.11	All the ETL code
Data processing	Pandas	Clean, join and transform the data
API access	requests library	Get the products from the REST API
Database	SQLite	Store the star schema
Query language	SQL	Create tables, views and indexes
Orchestration	Apache Airflow	Run and schedule the pipeline
Environment	WSL2 (Ubuntu) on Windows	Run Airflow and the pipeline
Version control	Git and GitHub	Store the code and documents

7.2 Deployment Diagram

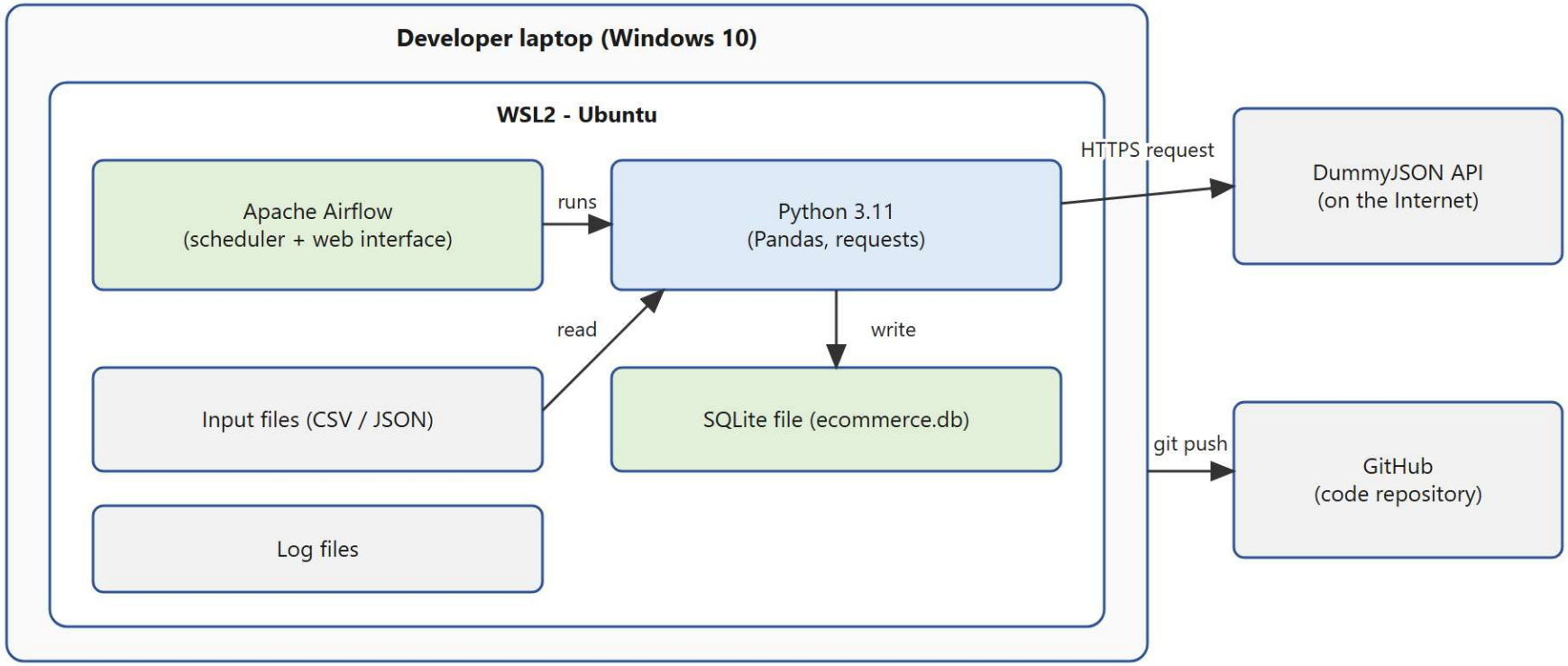


Figure 10: Deployment diagram

7.3 Component Diagram

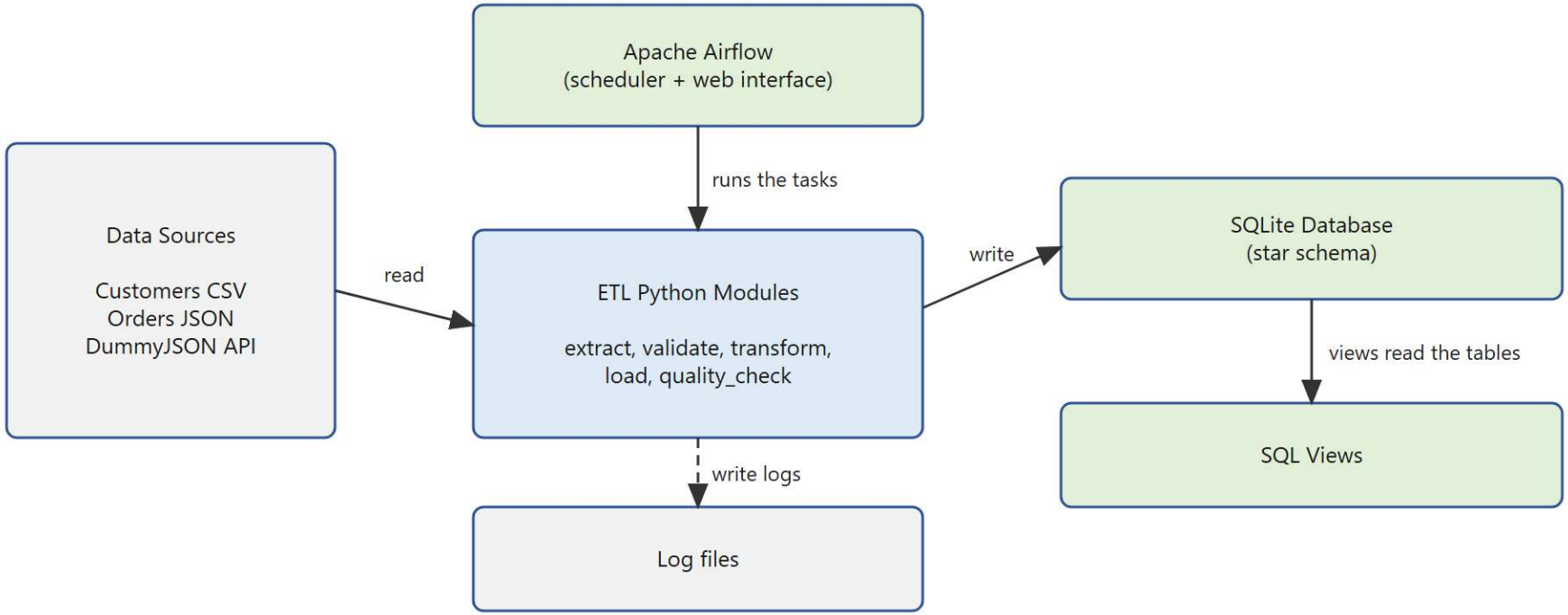


Figure 11: Component diagram

8. Testing & Validation

Test plan: the unit tests check one function at a time, the integration tests check the stages working together, and the user acceptance test checks the result as a data analyst would use it.

ID	Test	Action / input	Expected result	Type
T-01	Read customers file	Run the extract function on the CSV file	All rows are read into a DataFrame	Unit
T-02	Read orders file	Run the extract function on the JSON file	All records are read into a DataFrame	Unit
T-03	Get products from the API	Call the API	Status code 200 and a non-empty product list	Unit
T-04	API failure	Use a wrong API address	The task fails, is retried and the error is logged	Integration
T-05	Missing required column	Remove a required column from a test file	The Validate task fails with a clear message	Unit
T-06	Remove duplicates	Data with repeated rows	No duplicate rows after Transform	Unit
T-07	Handle missing values	Data with empty fields	No missing values in the key columns	Unit
T-08	Standardize formats	Dates in different formats	All dates are in YYYY-MM-DD	Unit
T-09	Load into the database	Run the Load task	Tables are created and the row counts match	Integration
T-10	Incremental loading	Run the pipeline twice on the same data	No repeated orders in fact_sales	Integration
T-11	SQL views	Query each view	Each view returns correct rows	Integration
T-12	Quality Check	Add a duplicate row by hand and run the check	The Quality Check task fails and reports the problem	Integration
T-13	Full DAG run	Trigger the DAG in Airflow	All five tasks are green	Integration
T-14	Analyst check	An analyst runs a view and checks the numbers	The numbers match the source data	User acceptance

9. Deployment Strategy

Hosting environment: the project runs on a local machine inside WSL2 (Ubuntu). It needs only free tools and no cloud account.

Steps to run the project:

- Download (clone) the project from GitHub.
- Install the Python packages from requirements.txt.
- Install and start Apache Airflow in WSL.
- Put the DAG file in the Airflow dags folder.
- Open the Airflow web interface and trigger the DAG.
- Check the run status and open the SQLite database to see the data.

Scaling considerations:

- The project uses sample data, so SQLite is enough. For much bigger data, SQLite can be replaced by a database server such as MySQL or PostgreSQL.
- The stages are separate modules, so a stage can be changed without touching the others.